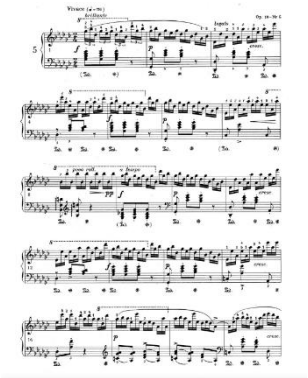

PROJET INFORMATIQUE POO – JAVA

« JAVAZIC »

I. Présentation. 2

II. Objectifs. 3

III. Organisation et planning. 4

IV. Cahier des charges détaillé. 4

1. Les fonctionnalités de base proposées aux différentes catégories d'utilisateurs. 5

2. Les fonctionnalités supplémentaires. 6

3. Architecture logicielle : modèle MVC. 7

4. Persistance des données : sauvegarde et chargement 10

V. Travail à réaliser, rendus et modalités d'évaluation 10

I. Présentation.

Le projet d'Informatique de ce deuxième semestre a pour objectif de concevoir et développer un programme de gestion d'un catalogue musical et de playlists, inspiré du fonctionnement des plateformes de streaming musical telles que Deezer ou Spotify.

Le programme doit permettre entre autres :

- La gestion d'un catalogue musical (morceaux, albums, artistes, groupes)
- La gestion de comptes utilisateurs
- La recherche et la sélection de morceaux ou d'albums
- La création et la gestion de playlists

...

Le projet sera intégralement développé en Java avec les bibliothèques standards, sans utiliser de frameworks externes. Il devra respecter les principes de la programmation orientée objet et une architecture MVC (Modèle – Vue – Contrôleur). Une première version du programme devra fonctionner intégralement en mode console. Une seconde version proposera une interface graphique (Swing ou JavaFX) reposant sur le même modèle.

Bien qu'il s'agisse d'un programme Java exécuté localement et non d'une application réellement déployée en environnement multi-utilisateurs, le système devra être conçu comme s'il pouvait être utilisé par plusieurs catégories d'acteurs distincts, disposant de droits et de responsabilités différents.

II. Objectifs.

À travers la mise en pratique des notions vues en cours de Programmation Orientée Objet, ce projet vise l'acquisition de plusieurs compétences :

- **Compétences en conception logicielle :**
  - Identifier les acteurs et les cas d'utilisation (Use Cases)
  - Modéliser en utilisant le langage UML :
    - Concevoir les diagrammes de cas d'utilisation
    - Concevoir les diagrammes de classes
  - Concevoir une architecture MVC avec une séparation stricte entre la logique métier (Model), l'interface graphique (View)et la gestion des interactions utilisateur (Controller).

- **Compétences techniques :**

Maîtriser les concepts de la programmation orientée objet :

- Encapsulation
- Héritage, polymorphisme, classes abstraites, interfaces
- Manipulation de collections (ArrayList, HashMap…)
- Sérialisation (sauvegarde / chargement)
- Gestion des exceptions

- **Compétences méthodologiques :**
  - Travailler en équipe.
  - Planifier un projet et respecter les délais impartis.
  - Utiliser des outils collaboratifs.
  - Elaborer et mettre en œuvre un plan de tests.

III. Organisation et planning.

- Le projet se conduit en équipe de trois ou quatre étudiants d'un même groupe de TD.
- Les étudiants doivent s'inscrire sur la plateforme projet la semaine du 16/03/2026.
- Le projet démarre la semaine du 23/03/2026 (distribution du sujet) et se termine le 19/04/2026 (rendu final déposé par chaque équipe).
- Des soutenances auront lieu la semaine du 20/04/2026.

16/03/2026	23/03/2026		19/04/2026	20/04/2026
Constitution des équipes	Distribution du sujet		Rendu final	Soutenances

IV. Cahier des charges détaillé.

A son lancement, le programme doit afficher un **menu principal** proposant de :

1. **Se connecter en tant qu'administrateur**
2. **Se connecter en tant que client**
3. **Créer un compte client**
4. **Continuer en tant que simple visiteur**
5. **Quitter**

Le programme fonctionne ensuite en mode interactif via des menus successifs.

Le système comporte au moins trois types d'utilisateurs, les administrateurs, les simples visiteurs et les abonnés. Pour chaque catégorie, votre programme devra proposer les fonctionnalités de base décrites ci-dessous. Vous devrez ensuite ajouter d'autres fonctionnalités à choisir parmi celles proposées plus loin.

1. Les fonctionnalités de base proposées aux différentes catégories d'utilisateurs.

A. Pour la catégorie Visiteur

Le visiteur peut :

- **Consulter le catalogue musical :**
  - Rechercher un morceau, un album, un artiste ou un groupe.
  - Consulter les détails d'un album, les informations sur un artiste ou un groupe.

  Le programme doit permettre de naviguer de manière ergonomique dans le catalogue :

  - A partir d'un morceau ou d'un album, consulter les informations sur l'artiste ou le groupe, découvrir les autres morceaux ou albums de ce groupe …
  - A partir d'un album, consulter les détails des morceaux.
  - A partir d'un groupe ou d'un artiste, avoir la liste des albums et/ou des morceaux, en sélectionner un, consulter ses détails …

Écouter un nombre limité de morceaux (par exemple 5 écoutes maximum par session). La lecture des morceaux de musique sera simulée par un temps de pause (en mode console) ou une barre de progression (en mode graphique.

Un simple visiteur ne peut pas créer de playlists et sauvegarder ses données.

B. Pour la catégorie Abonné

En plus de ce que peut déjà faire un simple visiteur, un abonné peut :

- **Créer et gérer des playlists :**
  - Créer, renommer ou supprimer une playlist.

- Ajouter un morceau dans une playlist. Le morceau pourra être choisi dans le catalogue selon différents critères de recherche ou dans une autre playlist.
- Retirer un morceau d'une playlist.

- **Écouter un nombre illimité de morceaux**
- **Consulter son historique d'écoute**

Les playlists d'un abonné et son historique d'écoute sont sauvegardés. Ils les retrouve à chaque connexion.

C. Pour la catégorie Administrateur

Le rôle d'un administrateur est de :

- **Gérer le catalogue musical :**
  - Ajouter ou supprimer des morceaux, des albums, des artistes ou des groupes.
- **Gérer les comptes des abonnés :**
  - Suspendre ou supprimer un compte abonné
- **Consulter des statistiques simples :**
  - Nombre d'utilisateurs
  - Nombre de morceaux, d'albums, de groupes …
  - Nombre total d'écoutes

2. Les fonctionnalités supplémentaires.

En plus des fonctionnalités de base, **vous devez proposer au moins deux fonctionnalités supplémentaires au choix** parmi celles décrites ci-dessous. Vous ne pouvez mettre en place ces fonctionnalités que si celles de base sont opérationnelles.

- **Système de notation et d'avis sur les morceaux.** Une note et/ou des commentaires peuvent être attribués et consultés par les abonnés. Un abonné peut modifier ou supprimer sa note ou son commentaire.
- **Recommandations** basées sur l'historique d'écoute et/ou sur le contenu des playlists

- **Statistiques évoluées** : nombre d'écoutes par morceaux, par album ou par artiste ; morceaux, albums ou artistes les plus écoutés, les plus fréquemment ajoutés dans les playlists …
- **Catalogue musical évolué avec classement** des morceaux selon différents critères (par style musical, par période …). **Recherche avancée** dans le catalogue avec **filtres**.
- **Playlist collaborative** : Le programme permet à un abonné de partager une playlist avec d'autres abonnés. Un abonné pourra ajouter un ou plusieurs collaborateurs à une playlist, les autoriser ou non à ajouter ou supprimer des morceaux. Seuls les abonnés peuvent être collaborateurs.

3. Architecture logicielle : modèle MVC.

Le programme devra être structuré selon l'architecture **MVC (Modèle – Vue – Contrôleur)**.

En réduisant les dépendances entre composants, l'architecture MVC offre une meilleure lisibilité du code, améliore son organisation, et facilite sa maintenance et son évolutivité (on peut changer la vue sans modifier le modèle).

Cette architecture repose sur une séparation claire des responsabilités entre trois couches :

- **Le modèle (Model)** contient la logique métier et les données du système (morceaux, utilisateurs, playlists, règles de gestion, calculs, etc.).

- **La vue (View)** gère l'affichage et détecte les interactions de l'utilisateur avec les différents composants graphiques (saisie clavier, clics souris).

- **Le contrôleur (Controller)** fait le lien entre le modèle et la vue : il interprète les actions de l'utilisateur et y réagit en appelant les méthodes du modèle et transmet les résultats à la vue.

Idéalement, le modèle doit être totalement indépendant des autres couches. Il ne doit contenir aucune dépendance vers la vue ou le contrôleur (aucune instruction d'affichage, aucune saisie utilisateur) :

Les relations entre les différentes couches doivent donc respecter les principes suivants :

- Le modèle ne connaît ni la vue ni le contrôleur.
- Le contrôleur connaît le modèle et la vue.
- La vue connaît le contrôleur.

Aucune autre relation ne devrait être autorisée. Autrement dit :

- Le modèle doit pouvoir fonctionner indépendamment de toute interface.
- Toute interaction entre le modèle et l'utilisateur doit passer par le contrôleur.

Chaque couche correspond bien entendu à un package distinct.

A. Le modèle (couche métier)

Le modèle contient toutes les classes qui permettent de stocker les données manipulées par le programme et qui regroupent les méthodes de traitements ou de calculs portant sur ces données. Le modèle contiendra en particulier les classes Catalogue, Morceau, Album, Artiste, Groupe, PlayList, Abonné …

Le modèle ne vous est pas totalement imposé mais vos choix devront être cohérents et réalistes. Vous devrez les justifier dans le rapport. Réfléchissez bien aux attributs qui caractérisent les objets et aux liens entre les objets :

Un morceau a un titre, une durée … Il peut être dans un ou plusieurs albums (album studio, albums live, compilations). Il est chanté par un artiste ou un groupe. Un artiste peut appartenir à un groupe …

B. La vue (affichage)

Le package vue regroupe toutes les classes avec les méthodes d'affichage : affichage des menus, des listes, des informations sur les objets …

Vous devrez concevoir deux systèmes d'affichage différents. Avec la mise en place du pattern MVC, la mise en place d'une seconde vue ne devrait pas nécessiter de modification du modèle.

Vue 1 : mode console

Une première version du programme doit fonctionner intégralement en mode console.

Vue 2 : mode graphique

Vous développerez ensuite une interface graphique Java avec les librairies Swing ou JavaFX.

C. Le contrôleur (gestion des actions utilisateur)

Les classes du package contrôleur ont pour rôle de déclencher les traitements ou calculs nécessaires pour répondre aux demandes de l'utilisateur, et de récupérer les données ou les résultats fournis par le modèle pour demander à la vue de les afficher.

Le Contrôleur fait le lien entre le modèle et la vue. Il interprète les actions de l'utilisateur, appelle les méthodes du modèle permettant de répondre à ses demandes, récupère les résultats et les transmet à la vue pour affichage.

4. Persistance des données : sauvegarde et chargement

Le programme devra permettre de sauvegarder et charger les données concernant :

- Les utilisateurs abonnés (informations du compte, playlists, historique …)
- Le catalogue musical (morceaux, albums, artistes ou groupes, …)

La sauvegarde pourra se faire au choix dans des fichiers textes ou par sérialisation. Les données seront automatiquement sauvegardées à la fermeture du programme.

V. Travail à réaliser, rendus et modalités d'évaluation

Vous devez appliquer la démarche de conception et de développement expliquée et vue en cours. Après avoir lu attentivement et étudié le cahier des charges (n'hésitez pas à poser des questions sur le CDC et à demander des précisions sur certains aspects), vous devez réfléchir pour concevoir une solution avant de passer aux phases de codage et de tests.

Le guide pas à pas ci-dessous constitue une recommandation méthodologique. Les équipes sont libres d'adapter leur démarche tant que les contraintes du cahier des charges sont respectées.

A. Guide pas à pas

1. Conception UML – logique métier

- **Etude des cas d'utilisation**
  - Identifier les différents acteurs et les éventuelles relations d'héritage entre acteurs.
  - Enumérer les principaux cas d'utilisation. Réfléchir aux relations entre cas d'utilisation (inclusion, extension, généralisation).

  - Etablir un **diagramme de cas d'utilisation** précis et détaillé.

- **Modèle de classes.**
  - Identifier les classes et leurs attributs.
  - Etablir les relations entre les classes. Pour les associations, réfléchir aux rôles de chaque entité, à la navigabilité et préciser les cardinalités.
  - Etablir un **diagramme de classes** précis et détaillé.

- **Description des cas d'utilisation en termes d'objets.**

  Chaque cas d'utilisation peut se dérouler suivant plusieurs **scénarios**. On distingue :
  - Le scénario principal, celui qui se produit le plus souvent, selon un déroulement « idéal », lorsque tout se passe correctement.
  - Les scénarios alternatifs qui représentent des variantes possibles du déroulement principal. Ces scénarios sont généralement rencontrés suite à des choix optionnels faits par l'utilisateur.
  - Les scénarios d'exception qui décrivent les situations d'erreur ou les situations pour lesquelles le déroulement normal ne peut aboutir (donnée invalide, droit insuffisant, contrainte non respectée …)

Avant de passer à l'implémentation, il faut décrire les scénarios (au moins les principaux) de manière chronologique en termes d'objets : quels sont les objets impliqués ? Quels objets faut-il créer ? Quelles méthodes doivent être appelées ?

Cette analyse vous permettra de vérifier que votre diagramme de classes permet effectivement de réaliser les cas d'utilisation. Elle vous aidera à préciser les méthodes nécessaires dans chaque classe.

2. Conception – Architecture MVC

Découpage en packages selon le modèle MVC

3. Implémentation du modèle et tests

La phase d'implémentation devra être progressive. Le programme devra être développé et validé étape par étape.

- Implémenter et tester chaque classe indépendamment pour vérifier la bonne initialisation des objets et le bon fonctionnement des méthodes.

- Etablir et mettre en œuvre un plan de tests pour valider le déroulement des différents cas d'utilisation. Ce **plan de tests** pourra prendre la forme d'un tableau précisant :
  - La fonctionnalité testée
  - Le scénario (principal, alternatif ou d'exception)
  - Les données d'entrée
  - Les résultats attendus
  - Les résultats obtenus

4. Conception – Interface graphique

- Faire des **maquettes** de l'interface graphique
- Etablir un **story-board**.

5. Implémentation d'une interface graphique swing ou JavaFX

B. Les rendus attendus

Le rapport

Vous devrez rendre un rapport contenant :

1. Planning suivi et répartition des tâches au sein de l'équipe.
2. Modélisation UML
   a. Diagramme de cas d'utilisation et explications
   b. Diagramme de classes et explications
   c. Description de quelques scénarios en termes d'interactions entre objets
3. Maquettes de l'interface graphique et story-board.
4. Plan de tests et mise en œuvre des tests
5. Bilans
6. Sources et références

Le programme

Votre programme devra être correctement indenté et commenté. Vous devrez remettre un dossier compressé contenant :

- Le projet complet compressé (sources organisées en packages).
- Un fichier *.jar* exécutable permettant de lancer le programme.
- La documentation générée (JavaDoc) au format HTML.
- Un fichier README précisant comment exécuter le programme et les éventuelles particularités.

C. Modalités d'évaluation

- Rapports
- Soutenances (la semaine du 20/04/2026)

L'évaluation du projet tiendra compte de votre travail collectif et individuel.

Travail collectif :

- Qualité des rendus et présentations.
- Pertinence et rigueur de la démarche de conception et de la modélisation.
- Qualité du code :
  - Sur la forme : indentation, commentaires, javadoc
  - Sur le fond : respect de l'architecture MVC, absence de classes « fourre-tout », inutiles ou artificielles, respect des principes de base de la POO.
- Respect du cahier des charges.
- Robustesse et gestion des erreurs(connexion avec identifiants incorrects, ajout d'un morceau déjà présent dans une playlist, suppression d'un élément inexistant …). Utilisation pertinente des exceptions.
- Créativité et originalité dans la limite des libertés offertes par le cahier des charges.
- Ergonomie et expérience utilisateur.

Travail individuel :

- Implication dans le projet, importances et difficultés des tâches réalisées.
- Qualité des réponses individuelles fournies au jury lors des soutenances.

Bon projet !

Julienne Palasi, Jean-Pierre Segado, Baligh Mnassri et toute l'équipe enseignante.